МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Санкт-Петербургский политехнический
университет Петра Великого»**
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01_03 Технологии машинного обучения и искусственного интеллекта

Отчёт по лабораторной работе № 1 по дисциплине «Дискретная математика»

«Операции над мультимножествами и генерация бинарного кода Грея»

Выполнил:
Студент группы 5130201/50302 _____ Калинский В.И.
Принял:
Преподаватель _____ Востров А.В.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Математическое описание **5**

1.1 Понятие множества, мультимножества и универсума 5

1.2 Теоретико-множественные и арифметические операции 6

1.3 Алгоритм генерации бинарного кода Грея 8

2 Особенности реализации **9**

2.1 Архитектура программного комплекса и структуры данных . . 9

2.2 Спецификация функций и методов 9

3 Результаты работы **14**

3.1 Сценарий 1. Тестирование краевого случая: пустой универсум ($n = 0$) 14

3.2 Сценарий 2. Тестирование защиты от некорректного пользовательского ввода 16

3.3 Сценарий 3. Генерация универсума и заполнение мультимножеств 17

3.4 Сценарий 4. Проверка теоретико-множественных операций (пункты меню 2–6) 18

3.5 Сценарий 5. Проверка арифметических операций (пункты меню 7–10) 20

3.6 Сценарий 6. Тестирование при большой разрядности и ограничение вывода 22

Заключение **23**

Список литературы. **24**

Приложение А. Заголовочный файл MultiSet.hpp **25**

Приложение Б. Файл реализации MultiSet.cpp **27**

Приложение В. Файл точки входа и интерфейса main.cpp **30**

Приложение Г. Скрипт сборки CMakeLists.txt **38**

Введение

В современной дискретной математике и информатике классическая теория множеств часто оказывается недостаточной для описания реальных информационных структур. Во многих задачах возникает необходимость оперировать наборами элементов, в которых один и тот же объект может присутствовать многократно. Такие математические объекты называются *мультимножествами*.

Для практического применения мультимножеств в вычислительных системах требуется определить конечное базовое множество(носитель) и универсальное мультимножество(универсум), относительно которого корректно определяются операции на мультимножествах. В качестве эффективного базиса для формирования универсума в данной работе выступает двоичный код Грея.

Цель работы: изучение алгебраических и теоретико-множественных свойств мультимножеств, освоение алгоритмов генерации перестановок и кодов Грея, а также программная реализация консольного приложения для моделирования операций над мультимножествами.

Постановка задачи:

1. Разработать алгоритм и программный модуль генерации n-разрядного бинарного кода Грея, где разрядность $n \in [0, 27]$ задается пользователем. Верхняя граница разрядности выбрана исходя из критерия времени генерации до 5 секунд (при $n = 28$ среднее время составляет 7.3 с, при $n = 27$ – 3.6 с). Предусмотреть обработку краевого случая пустого универсума при $n = 0$.

2. Сформировать упорядоченный универсум U, кратности элементов которого генерируются случайным образом в диапазоне от 1 до 10 (при $n = 0$ универсум пуст).

3. Реализовать создание двух мультимножеств A и B над построенным носителем с мощностями, задаваемыми пользователем в диапазоне от 0 до доступной суммарной емкости универсума (не более 100).

4. Предусмотреть два альтернативных режима наполнения мультимножеств:

   - ручной ввод пользователем (выбор номера элемента носителя и задание его кратности с контролем непревышения как целевой мощности, так и доступной емкости элемента в универсуме);
   - автоматическая генерация кратностей элементов на основе датчика псевдослучайных чисел с контролем лимитов универсума.

5. Реализовать вычисление следующих операций над мультимножествами:

   - объединение ($A \cup B$);
   - пересечение ($A \cap B$);
   - разность в обоих направлениях ($A \setminus B$ и $B \setminus A$);
   - симметрическая разность ($A \triangle B$);
   - дополнение до универсума ($\overline{A} = U \setminus A$ и $\overline{B} = U \setminus B$);
   - арифметическая сумма ($A + B$);
   - арифметическая разность ($A - B$ и $B - A$);
   - арифметическое произведение ($A \cdot B$);
   - арифметическое деление ($A : B$ и $B : A$).

6. Предусмотреть ограничение вывода мультимножеств в консоль, строгую фильтрацию некорректного пользовательского ввода и интерактивное меню управления.

1 Математическое описание

1.1 Понятие множества, мультимножества и универсума

Классическим *множеством* называется совокупность определенных, различимых объектов (элементов), мыслимая как единое целое, причем каждый элемент либо принадлежит множеству, либо не принадлежит ему.

Мультимножеством над базовым множеством $X = \{x_1, x_2, \ldots, x_N\}$ называется пара $\hat{A} = (X, k_A)$, где $k_A : X \to \mathbb{N} \cup \{0\}$ – характеристическая функция кратности, сопоставляющая каждому элементу $x_i \in X$ целое неотрицательное число $\alpha_i \geq 0$, называемое *кратностью* (степенью вхождения) элемента в мультимножество.

Символически мультимножество записывается в виде:

$$\hat{A} = \{x_1^{\alpha_1}, x_2^{\alpha_2}, \ldots, x_N^{\alpha_N}\}, \quad \alpha_i \in [0, k], \tag{1}$$

где $x_i^{\alpha_i}$ обозначает вхождение элемента x_i ровно α_i раз. Если $\alpha_i = 0$, элемент фактически отсутствует в мультимножестве. Пустое мультимножество обозначается вектором нулевых кратностей: $\varnothing = (0, 0, \ldots, 0)$.

Мощностью мультимножества $|\hat{A}|$ называется сумма кратностей всех его элементов:

$$|\hat{A}| = \sum_{i=1}^{N} k_A(x_i) = \sum_{i=1}^{N} \alpha_i. \tag{2}$$

Универсумом мультимножеств U называется фиксированное мультимножество над тем же носителем X:

$$U = \{x_1^{m_1}, x_2^{m_2}, \ldots, x_N^{m_N}\}, \quad m_i \geq 0, \tag{3}$$

ограничивающее сверху максимальную кратность элементов любого рассматриваемого мультимножества: $\forall i \ (\alpha_i \leq m_i)$.

В рамках разработанной программы универсум U формируется следующим образом: кратности его элементов m_i задаются псевдослучайными числами в диапазоне от 1 до 10:

$$U = \{x_1^{m_1}, x_2^{m_2}, \ldots, x_N^{m_N}\}, \quad m_i \in [1, 10]. \tag{4}$$

В краевом случае нулевой разрядности кода Грея ($n = 0$) носитель элементов отсутствует, универсум является пустым ($U = \varnothing$, $|U| = 0$), а мультимножества A и B автоматически инициализируются как пустые.

1.2 Теоретико-множественные и арифметические операции

Пусть $A = \{x_i^{\alpha_1}\}$ и $B = \{x_i^{\alpha_2}\}$ – два мультимножества над носителем X, а $U = \{x_i^{m_i}\}$ – универсум с кратностями $m_i \in [1, 10]$. Правила вычисления кратностей результирующих мультимножеств определяются следующим образом:

1. **Пересечение ($A \cap B$):** Кратность каждого элемента равна минимальной из кратностей в операндах:

$$k_{A \cap B}(x_i) = \min(\alpha_{1,i}, \alpha_{2,i}). \tag{5}$$

2. **Объединение ($A \cup B$):** Кратность каждого элемента равна максимальной из кратностей в операндах:

$$k_{A \cup B}(x_i) = \max(\alpha_{1,i}, \alpha_{2,i}). \tag{6}$$

3. **Дополнение ($\overline{A}$ и $\overline{B}$):** Дополнение мультимножества вычисляется относительно универсума U:

$$k_{\overline{A}}(x_i) = m_i - \alpha_{1,i}, \quad k_{\overline{B}}(x_i) = m_i - \alpha_{2,i}. \tag{7}$$

4. **Разность ($A \setminus B$ и $B \setminus A$):** В теоретическом конспекте разность определяется через пересечение с дополнением: $A \setminus B = A \cap \overline{B}$. Подставляя формулу дополнения, получаем:

$$k_{A \setminus B}(x_i) = \min(\alpha_{1,i}, m_i - \alpha_{2,i}). \tag{8}$$

Аналогично, для обратной разности:

$$k_{B \setminus A}(x_i) = \min(\alpha_{2,i}, m_i - \alpha_{1,i}). \tag{9}$$

5. **Симметрическая разность ($A \triangle B$):** Определяется как разность между объединением и пересечением мультимножеств:

$$A \triangle B = (A \cup B) \setminus (A \cap B) = (A \cup B) \cap \overline{(A \cap B)}. \tag{10}$$

Кратность элемента:

$$k_{A \triangle B}(x_i) = \min\Big(\max(\alpha_{1,i}, \alpha_{2,i}),\ m_i - \min(\alpha_{1,i}, \alpha_{2,i})\Big). \tag{11}$$

6. **Арифметическая сумма ($A + B$):** Кратность каждого элемента в сумме мультимножеств ограничивается сверху его максимальной кратностью в универсуме U:

$$k_{A+B}(x_i) = \min(\alpha_{1,i} + \alpha_{2,i},\ m_i). \tag{12}$$

7. **Арифметическая разность ($A - B$ и $B - A$):** Кратность представляет собой усеченную разность (не может быть отрицательной):

$$k_{A-B}(x_i) = \max(0, \alpha_{1,i} - \alpha_{2,i}), \tag{13}$$

$$k_{B-A}(x_i) = \max(0, \alpha_{2,i} - \alpha_{1,i}). \tag{14}$$

8. **Арифметическое произведение ($A \cdot B$):** Поэлементное произведение кратностей с обязательным ограничением сверху кратностью универсума U:

$$k_{A \cdot B}(x_i) = \min(\alpha_{1,i} \cdot \alpha_{2,i}, m_i). \tag{15}$$

9. **Арифметическое деление ($A : B$ и $B : A$):** Целочисленное деление кратностей. В соответствии с договоренностью при делении на 0 результирующая кратность принимается равной 0:

$$k_{A/B}(x_i) = \begin{cases} \alpha_{1,i}/\alpha_{2,i}, & \alpha_{2,i} \neq 0, \\ 0, & \alpha_{2,i} = 0. \end{cases} \tag{16}$$

$$k_{B/A}(x_i) = \begin{cases} \alpha_{2,i}/\alpha_{1,i}, & \alpha_{1,i} \neq 0, \\ 0, & \alpha_{1,i} = 0. \end{cases} \tag{17}$$

1.3 Алгоритм генерации бинарного кода Грея

Бинарным кодом Грея порядка n называется последовательность всех 2^n двоичных векторов длины n, упорядоченная таким образом, что любые два соседних кода, а также первый и последний, отличаются ровно на один разряд.

Для генерации последовательности кодовых слов используется алгоритм, описанный в фундаментальном учебнике Ф. А. Новикова «Дискретная математика для программистов» [1]. Согласно данному алгоритму, переход от кодового слова B_{i-1} к следующему кодовому слову B_i $(i = 1, 2, \ldots, 2^n - 1)$ осуществляется инверсией бита с индексом $p = Q(i) - 1$, где функция $Q(i)$ возвращает номер младшей единицы в двоичной записи шага i:

$$Q(i) = 1 + \max\{k \in \mathbb{N} \cup \{0\} \mid i \text{ делится на } 2^k\}. \tag{18}$$

Алгоритмически вычисление $Q(i)$ представляет собой подсчет числа делений i на 2 до получения нечетного остатка:

$$\begin{cases} q = 1, \quad j = i; \\ \text{пока } (j \pmod 2 == 0): \quad j = j/2, \quad q = q + 1; \\ \text{возврат } q. \end{cases} \tag{19}$$

Тогда закон формирования кодовой последовательности имеет вид:

$$B_0 = 0, \quad B_i = B_{i-1} \oplus (1 \ll (Q(i) - 1)), \quad i = 1, \ldots, 2^n - 1, \tag{20}$$

где $\oplus$ – побитовая операция XOR, $\ll$ – побитовый сдвиг влево. Вектор сгенерированных целых чисел служит базисом упорядоченного универсума.

Разрядность генерируемого кода ограничена диапазоном $n \in [0, 27]$. Выбор верхней границы обусловлен временным критерием: время построения носителя не должно превышать 5 секунд. В ходе экспериментальных замеров при $n = 28$ среднее время работы генератора составило 7.3 секунды, тогда как при $n = 27$ оно составило 3.6 секунды, что удовлетворяет установленному критерию быстродействия. При $n = 0$ генерация не выполняется, и универсум остаётся пустым.

2 Особенности реализации

2.1 Архитектура программного комплекса и структуры данных

Программный комплекс спроектирован на языке C++ (стандарт C++20) в соответствии с принципами модульности и разделения ответственности:

- `MultiSet.hpp` – спецификация интерфейса класса мультимножества и внешних операторов;
- `MultiSet.cpp` – реализация операций над мультимножествами с применением алгоритмов стандартной библиотеки шаблонов STL (`std::transform`);
- `main.cpp` – модуль консольного взаимодействия, генерации кода Грея, ввода данных и управления логикой;
- `CMakeLists.txt` – сценарий автоматизированной сборки проекта со статической линковкой библиотеки `mylib` и интеграцией динамического санитайзера памяти AddressSanitizer (`-fsanitize=address -fno-omit-frame-pointer -O1`).

Основной структурой данных для представления мультимножества выступает класс `MultiSet`, являющийся адаптером *std::vector<int>*. Внутреннее состояние объекта описывается вектором кратностей `std::vector<int> counts`.

Длина вектора равна текущей мощности носителя универсума $N = 2^{\text{bitrate}}$ (переменная `U_SIZE`). Индекс ячейки вектора i однозначно сопоставлен i-му кодовому слову кода Грея. Глобальный универсум хранится в переменной `MultiSet multi_universe`.

2.2 Спецификация функций и методов

В таблице 1 приведено исчерпывающее описание всех функций и методов программного комплекса с указанием их входных и выходных параметров.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`MultiSet::MultiSet()`	Инициализация параметров программы	Инициализированный объект мультимножества размера `U_SIZE`, заполненный нулями.
`MultiSet::begin()`	Инициализация параметров программы	Итератор `std::vector<int>::iterator` на первый элемент вектора кратностей.
`MultiSet::end()`	Инициализация параметров программы	Итератор `std::vector<int>::iterator` на конец вектора кратностей.
`MultiSet::size()`	Инициализация параметров программы	Целое число типа `int` – размер вектора кратностей.
`int& MultiSet::operator[] (size_t pos)`	Позиция элемента `pos` типа `size_t`	Ссылка `int&` на кратность элемента по индексу `pos`.
`void MultiSet::resize (size_t new_size, const int val)`	Новый размер `new_size`, значение заполнения `val`	Модификация размера внутреннего контейнера.
`int Q(int i)`	Целое число $i \geq 1$ (шаг генерации)	Целое число $q \geq 1$ – номер изменяемого разряда в коде Грея.
`void generateGrayCode (int n, vector<int> &target)`	Разрядность $n \in [0, 27]$, ссылка на вектор `target`	Заполнение вектора 2^n кодовыми словами Грея.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`void printBinary (int code, int bitrate)`	Число `code`, разрядность `bitrate`	Побитовый вывод числа в консоль.
`void printMultiSet (MultiSet &ms, const vector<int> &u, int bitrate)`	Ссылка на мультимножество `ms`, вектор универсума `u`, разрядность `bitrate`	Форматированный вывод пар вида `'код': кратность` (при $\|u\| > 32$ выводятся первые 32 элемента).
`long long createUniverse (MultiSet &u)`	Ссылка на универсум `u`	Число типа `long long` – суммарная ёмкость универсума; заполнение кратностей случайными числами от 1 до 10.
`int getValidatedInt (const string &prompt, int min_val, int max_val)`	Приглашение ввода, границы диапазона `min_val`, `max_val`	Целое число типа `int`, прошедшее валидацию диапазона и типа.
`void fillManually (MultiSet &ms, const vector<int> &universe, int bitrate, int target_capacity)`	Ссылка на множество `ms`, вектор универсума, разрядность, целевая мощность	Поэлементное ручное заполнение с контролем доступной ёмкости каждого элемента в универсуме.
`void fillAutomatically (MultiSet &ms, const vector<int> &universe, int target_capacity)`	Ссылка на множество `ms`, вектор универсума, целевая мощность	Случайное распределение кратностей с контролем лимитов кратностей универсума.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`MultiSet uni (MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – объединение $A \cup B$ ($\max(a_i, b_i)$).
`MultiSet insertion (MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – пересечение $A \cap B$ ($\min(a_i, b_i)$).
`MultiSet complement (MultiSet &A)`	Ссылка на мультимножество A	Объект `MultiSet` – дополнение $\overline{A} = U \setminus A$.
`MultiSet difference (MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – разность $A \setminus B = A \cap \overline{B}$.
`MultiSet sym_-difference (MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – симметрическая разность $(A \cup B) \setminus (A \cap B)$.
`MultiSet operator+ (MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – арифметическая сумма $\min(a_i + b_i, m_i)$.
`MultiSet operator- (MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – арифметическая разность $A - B$.
`MultiSet operator* (MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – произведение $A \cdot B$, ограниченное U.
`MultiSet operator/ (MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – результат деления $A : B$ (с защитой от нуля).

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`int main (int argc, char const *argv[])`	Инициализация параметров программы (`argc`, `argv`)	Код возврата операционной системы `0`.

Полный исходный код каждого файла проекта вынесен в отдельные приложения:

- заголовочный файл `MultiSet.hpp` – в Приложении А (см. листинг 1);
- файл реализации операций `MultiSet.cpp` – в Приложении Б (см. листинг 2);
- файл главного модуля и TUI `main.cpp` – в Приложении В (см. листинг 3);
- скрипт системы сборки `CMakeLists.txt` – в Приложении Г (см. листинг 4).

3 Результаты работы

Для проверки разработанного программного комплекса проведены серии тестов, охватывающие как штатную обработку теоретико-множественных и арифметических операций, так и проверку отказоустойчивости при некорректном вводе данных и обработку граничных состояний (краевой случай нулевой разрядности, предельная разрядность носителя).

3.1 Сценарий 1. Тестирование краевого случая: пустой универсум ($n = 0$)

Пользователь задает нулевую разрядность кода Грея ($n = 0$). В этом случае базовый носитель элементов отсутствует ($N = 2^0 = 0$ кодов в логике программы), поэтому универсум является пустым. Программа выводит диагностическое сообщение: «`Универсум пуст (мощность равна 0). Множества A и B автоматически оставлены пустыми.`» и сразу переходит к главному меню без запроса мощностей и режимов заполнения.

При выборе пункта меню 1 на экран выводится статус отсутствия элементов во всех трех множествах. Выполнение любой из операций (пункты меню 2–10) не приводит к аварийному завершению программы или ошибкам сегментации (SEGV): циклы преобразования по нулевому диапазону корректно завершаются, и в качестве результата выводится пустое мультимножество $\varnothing$.

На рис. 1 представлен вывод консоли при вводе разрядности 0: зафиксированы сообщение об автоматической инициализации пустых множеств A, B и универсума, а также корректное отображение пустого состояния на примере вызова второго пункта меню без сбоев адресного пространства.

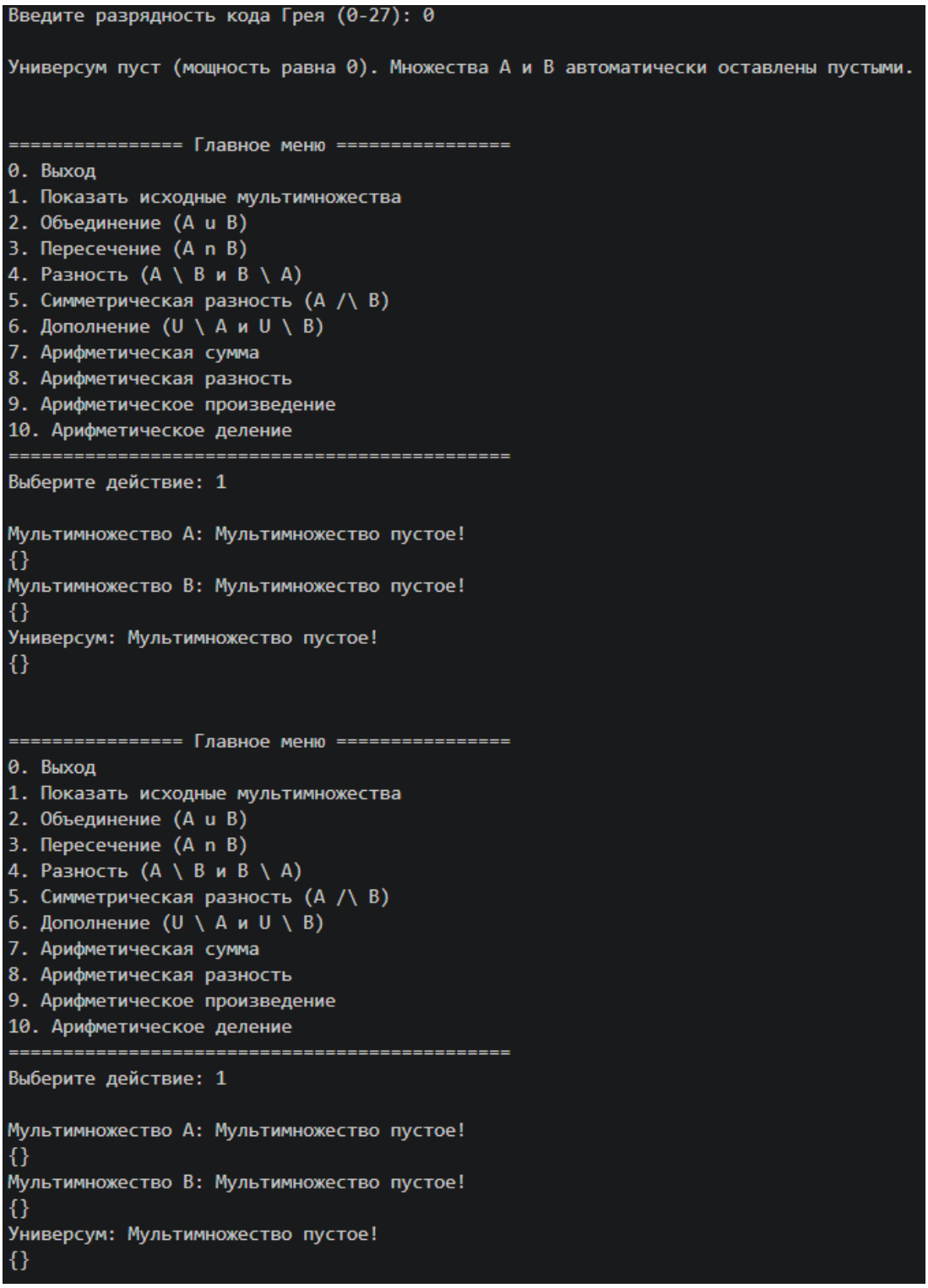


Рис. 1: Работа программы в режиме нулевой разрядности кода Грея ($n = 0$)

3.2 Сценарий 2. Тестирование защиты от некорректного пользовательского ввода

В рамках сценария проверяется реакция подсистемы валидации `getValidatedInt()`:

1. Ввод нечисловых символов (`"abc"`, `"!?"`) на этапе выбора разрядности;
2. Ввод отрицательного числа (-5 при допустимом минимуме 0);
3. Ввод числа, превышающего допустимый диапазон быстродействия (28 или 35 при максимуме 27);
4. Попытка задать кратность элемента при ручном заполнении, превышающую оставшуюся емкость данного элемента в сгенерированном универсуме U.

При нарушении формата потока ввода `cin` программа очищает флаги ошибок вызовом `cin.clear()`, освобождает входной буфер методом `cin.ignore()` и выводит сообщение с указанием корректных границ допустимого диапазона, предотвращая зацикливание.

```
Введите разрядность кода Грея (0-27): abc
Ошибка ввода! Пожалуйста, введите корректное число от 0 до 27.
Введите разрядность кода Грея (0-27): !?
Ошибка ввода! Пожалуйста, введите корректное число от 0 до 27.
Введите разрядность кода Грея (0-27): -5
Ошибка ввода! Пожалуйста, введите корректное число от 0 до 27.
Введите разрядность кода Грея (0-27): 28
Ошибка ввода! Пожалуйста, введите корректное число от 0 до 27.
Введите разрядность кода Грея (0-27): 3
Количество кодов: 8. Прошло времени: 9.0545e-05 сек.

Сгенерированный универсум U: {'000': 8, '001': 6, '011': 5, '010': 5, '110': 3, '111': 5,
 '101': 2, '100': 4}

Введите мощность мультимножества A (от 0 до 38): 100
Ошибка ввода! Пожалуйста, введите корректное число от 0 до 38.
```

Рис. 2: Отработка защиты от некорректного пользовательского ввода

На рис. 2 показана реакция терминала: ввод символьных строк и чисел вне диапазона $[0, 27]$ блокируется с выводом диагностических предупреждений и повторным запросом ввода до получения корректного значения.

3.3 Сценарий 3. Генерация универсума и заполнение мультимножеств

Пользователь задает разрядность $n = 3$. Программа генерирует $N = 2^3 = 8$ комбинаций бинарного кода Грея. Пользователь задает мощности: $|A| = 5$ (ручное заполнение), $|B| = 8$ (автоматическое заполнение). При ручном заполнении множества A интерфейс отображает доступную кратность элементов в U и динамически ограничивает максимальный ввод. Множество B наполняется автоматически функцией `fillAutomatically()` со случайным распределением кратностей с обязательным контролем наличия элементов в универсуме.

На рис. 3 показан вывод консоли при генерации 8 кодов Грея, случайном задании кратностей универсума, а также пошаговый процесс ручного ввода множества A с контролем лимитов емкости U и автоматической генерации множества B. Полученные мультимножества A и B показаны при вызове пункта 1 меню.

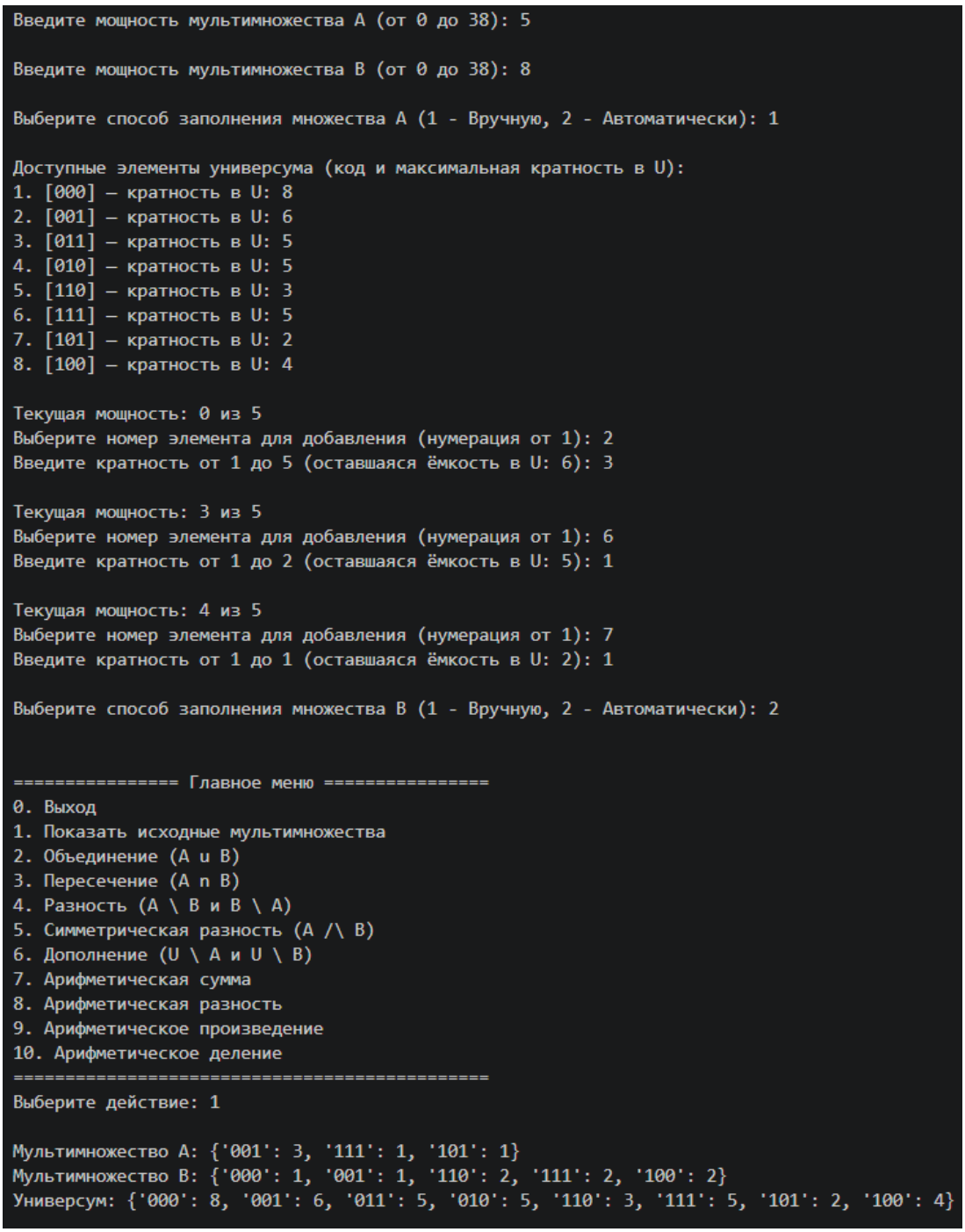


Рис. 3: Протокол генерации универсума и наполнения мультимножеств A и B

3.4 Сценарий 4. Проверка теоретико-множественных операций (пункты меню 2–6)

На полученных мультимножествах из предыдущих сценариев протестированы операции из пунктов меню 2–6. Полученные результаты строго согласуются с ручным теоретическим расчетом по формулам, приведенным

в подразделе 1.2. На рис. 4 зафиксированы экранные снимки консоли при вызове пунктов меню 2–4, а на рис. 5 – пунктов 5 и 6 меню: для каждого пункта отображается правильный результат.

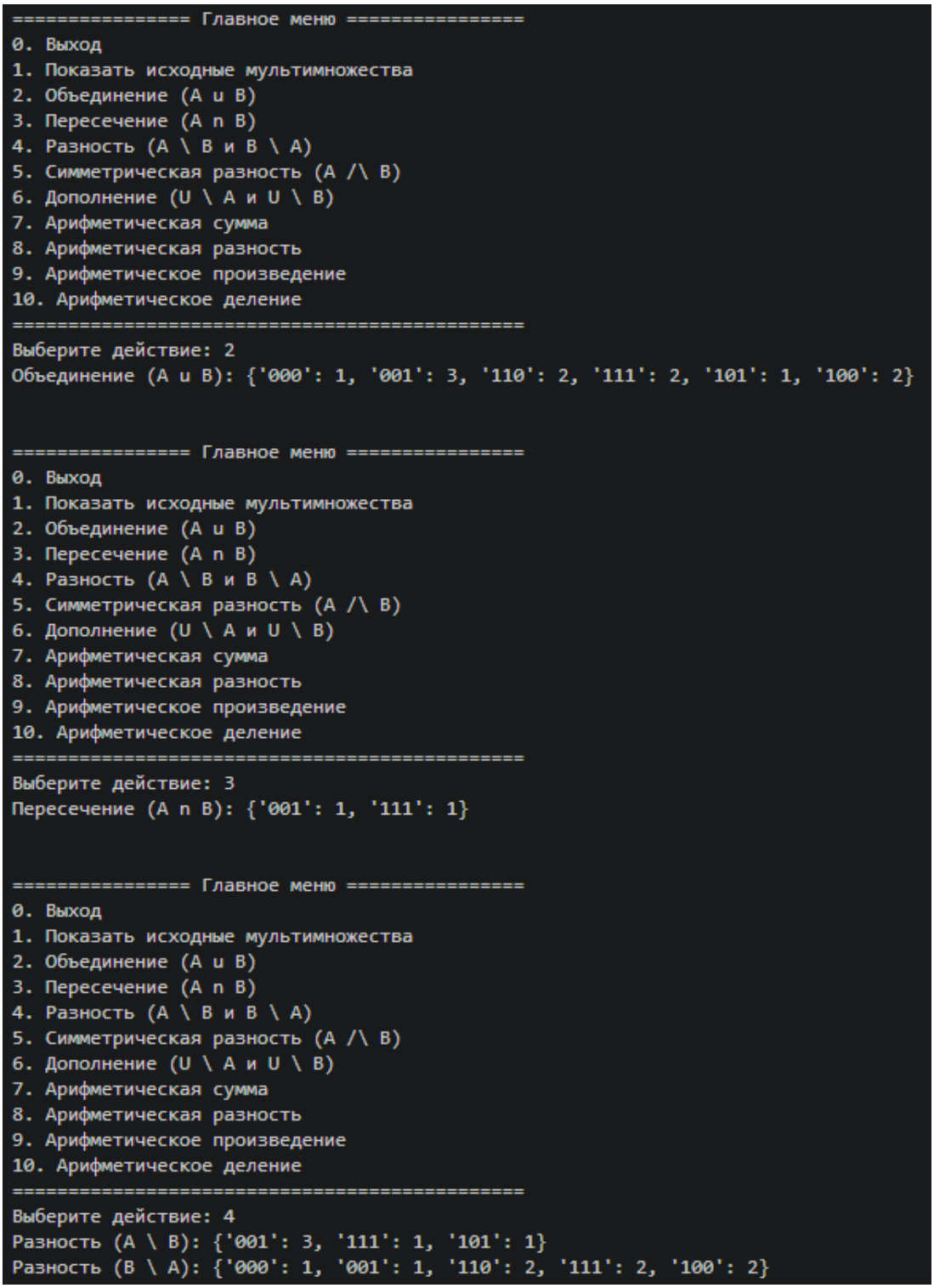


Рис. 4: Результаты выполнения теоретико-множественных операций (пункты меню 2–4)

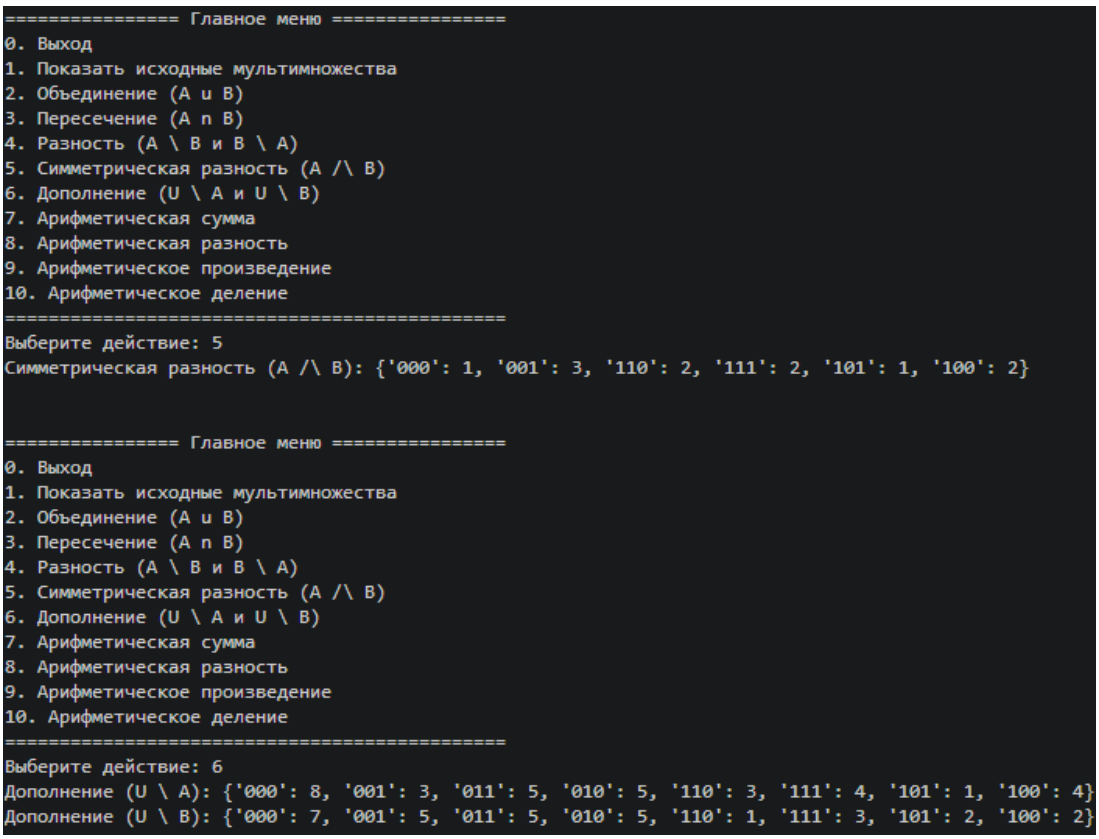


Рис. 5: Результаты выполнения теоретико-множественных операций (пункты меню 5 и 6)

3.5 Сценарий 5. Проверка арифметических операций (пункты меню 7–10)

На полученных мультимножествах из предыдущих сценариев протестированы операции из пунктов меню 7–10. Полученные результаты строго согласуются с ручным теоретическим расчетом по формулам, приведенным в подразделе 1.2. На рис. 6 зафиксированы экранные снимки консоли при вызове пунктов меню 7–10: для каждого пункта отображается правильный результат.

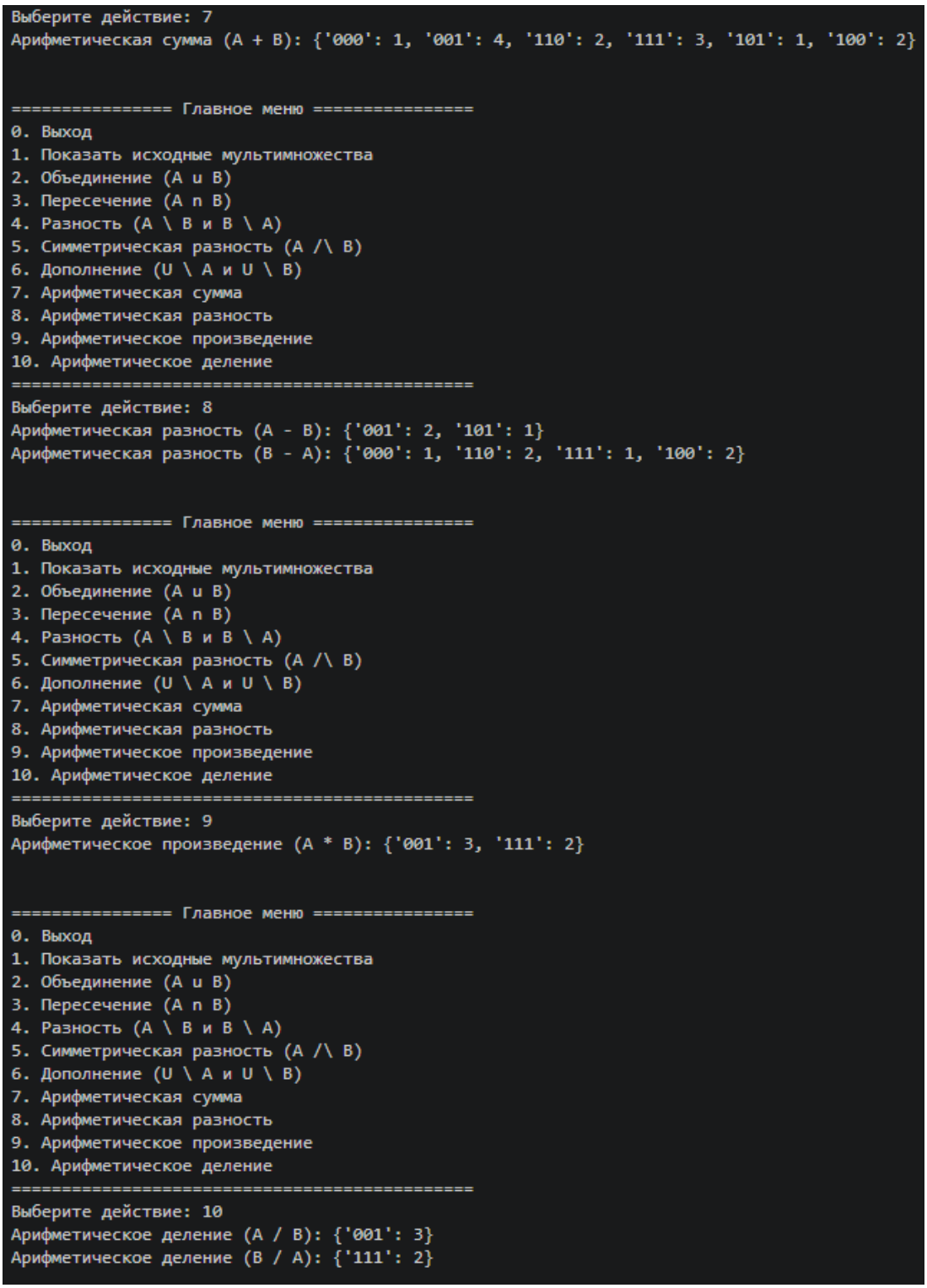


Рис. 6: Результаты выполнения арифметических операций (пункты меню 7–10)

3.6 Сценарий 6. Тестирование при большой разрядности и ограничение вывода

Для проверки масштабируемости программы проведен запуск с повышенной разрядностью. При предельной разрядности $n = 27$ размер носителя составляет $N = 2^{27} = 134\,217\,728$ кодовых комбинаций. Зафиксированное время генерации составило 3.72468 секунды, что удовлетворяет требованию быстродействия до 5 секунд.

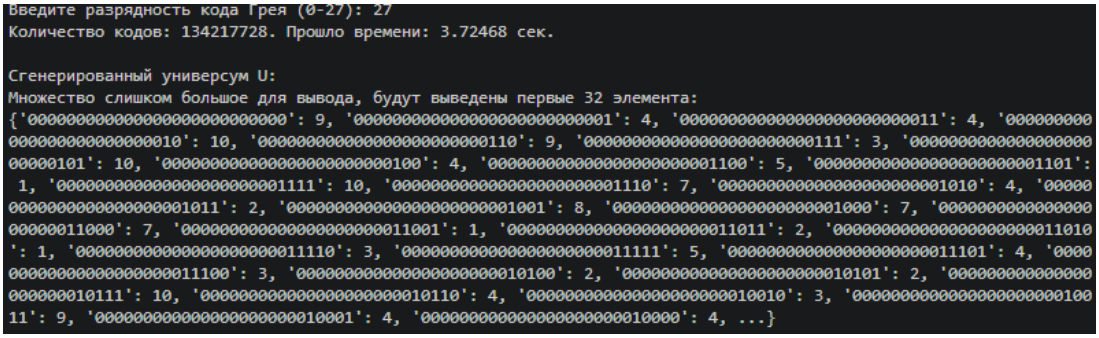


Рис. 7: Ограничение вывода до 32 элементов и замер времени генерации при больших n

На рис. 7 продемонстрирована работа механизма усечения вывода для размерностей более 32 элементов, а также замер времени работы генератора кода Грея при $n = 27$.

Заключение

В ходе лабораторной работы на языке C++20 разработан программный комплекс для генерации бинарного кода Грея и выполнения полного набора операций над мультимножествами. В рамках реализации спроектирован класс `MultiSet` на базе вектора кратностей, поддержаны ручной и автоматический режимы ввода, независимая генерация кратностей элементов универсума случайными числами от 1 до 10, а также корректная обработка краевого случая пустого универсума ($n = 0$). По критерию допустимого времени генерации (≤ 5 с) максимальная разрядность кода Грея зафиксирована равной $n = 27$ со средним временем построения, равном 3.6 секунды. Реализованы все требуемые теоретико-множественные и арифметические операции, обеспечена строгая валидация пользовательского ввода и проведено тестирование, подтвердившее надежность и корректность выполнения программы.

Достоинства реализованного решения:

- Высокая скорость доступа к элементам и линейная сложность вычисления операций над мультимножеством.
- Лаконичность и безопасность кода благодаря алгоритмам STL (`std::transf`

Недостатки решения:

- Экспоненциальный расход памяти при росте разрядности кода Грея ($N = 2^n$).
- Неэффективность плотного вектора для сильно разреженных мультимножеств.

Масштабирование алгоритма: Для работы с большими универсумами ($n \geq 30$) целесообразно заменить плотный вектор ассоциативным контейнером `unordered_map<int, int>`, реализованный на хэш-таблице и не хранящей нулевых элементов. Также для ускорения поэлементной обработки возможно применение параллельных вычислений.

Список литературы

[1] Новиков Ф. А. Дискретная математика для программистов : Учебник для вузов / Ф. А. Новиков. – 3-е изд. – СПб. : Питер, 2009. – 384 с. (Дата обращения – 10.10.2026)

Приложение А. Заголовочный файл MultiSet.hpp

Листинг 1: Заголовочный файл MultiSet.hpp

```
#ifndef SET_HPP
#define SET_HPP

#include <vector>

class MultiSet
{
    std::vector<int> counts;
    public:
    MultiSet();
    std::vector<int>::iterator begin()
    {
        return counts.begin();
    }
    int size()
    {
        return counts.size();
    }
    std::vector<int>::iterator end()
    {
        return counts.end();
    }
    int &operator[](size_t pos)
    {
        return counts[pos];
    }
    void resize(size_t new_size, const int val){
        counts.resize(new_size, val);
    }
};

extern int U_SIZE;
extern MultiSet multi_universe;

MultiSet uni(MultiSet &A, MultiSet &B);
MultiSet insertion(MultiSet &A, MultiSet &B);
MultiSet difference(MultiSet &A, MultiSet &B);
MultiSet sym_difference(MultiSet &A, MultiSet &B);
MultiSet complement(MultiSet &A);

MultiSet operator+(MultiSet &A, MultiSet &B);
MultiSet operator-(MultiSet &A, MultiSet &B);
MultiSet operator*(MultiSet &A, MultiSet &B);
```

```
44        MultiSet operator/(MultiSet &A, MultiSet &B);
45
46        #endif
```

Приложение Б. Файл реализации MultiSet.cpp

Листинг 2: Файл реализации MultiSet.cpp

```cpp
#include "MultiSet.hpp"
#include <algorithm>
#include <cmath>
#include <iostream>

using namespace std;

MultiSet::MultiSet() : counts(U_SIZE, 0) {}

// объединение: max(a, b)
MultiSet uni(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return max(a, b); });
    return result;
}

// пересечение: min(a, b)
MultiSet insertion(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return min(a, b); });
    return result;
}

// дополнение: m - a, m - кратностьэлта- вуниверсуме
MultiSet complement(MultiSet &A)
{
    MultiSet result;
    transform(A.begin(), A.end(), multi_universe.begin(), result.begin(), [](int a, int
        m)
    { return m - a; });
    return result;
}

// разность: A n -B
MultiSet difference(MultiSet &A, MultiSet &B)
{
    MultiSet notB = complement(B);
    return insertion(A, notB);
}
```

```
43
44      // симметрическаяразность: (A u B) \ (A n B)
45      MultiSet sym_difference(MultiSet &A, MultiSet &B)
46      {
47          MultiSet AuB = uni(A, B);
48          MultiSet AnB = insertion(A, B);
49          return difference(AuB, AnB);
50      }
51
52      // арифметическаясумма
53      MultiSet operator+(MultiSet &A, MultiSet &B)
54      {
55          MultiSet result;
56          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
57          { return a + b; });
58
59          transform(result.begin(), result.end(), multi_universe.begin(), result.begin(), [](
                int sum, int m)
60          { return min(sum, m); });
61          return result;
62      }
63
64      // арифметическаяразность
65      MultiSet operator-(MultiSet &A, MultiSet &B)
66      {
67          MultiSet result;
68          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
69          { return max(0, a - b); });
70          return result;
71      }
72
73      // арифметическоепроизведение
74      MultiSet operator*(MultiSet &A, MultiSet &B)
75      {
76          MultiSet result;
77          for (int i = 0; i < U_SIZE; i++)
78          {
79              result[i] = min(A[i] * B[i], multi_universe[i]);
80          }
81          return result;
82      }
83
84      // арифметическоеделение
85      MultiSet operator/(MultiSet &A, MultiSet &B)
86      {
87          MultiSet result;
```

```
88          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
89          { return (b == 0) ? 0 : (a / b); });
90          return result;
91      }
```

Приложение В. Файл точки входа и интерфейса main.cpp

Листинг 3: Файл точки входа и интерфейса main.cpp

```
#include <string>
#include <limits>
#include <iostream>
#include <vector>
#include <bitset>
#include <format>
#include <algorithm>
#include <chrono>
#include "MultiSet.hpp"

int U_SIZE = 0;
MultiSet multi_universe;

using namespace std;
int getValidatedInt(const string &prompt = "", int min_val = 0, int max_val =
    numeric_limits<int>::max())
{
    int value;
    while (true)
    {
        cout << prompt;
        if (cin >> value && value >= min_val && value <= max_val)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Ошибка ввода! Пожалуйста, введитекорректноечислоот" << min_val << " до"
            << max_val << ".\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int Q(int i)
{
    int q = 1;
    int j = i;
    while (j % 2 == 0)
    {
        j = j / 2;
        q = q + 1;
    }
    return q;
```

```
42      }
43
44      void generateGrayCode(int n, vector<int> &target)
45      {
46          target.clear();
47          int B = 0;
48          target.push_back(B);
49
50          int total_steps = 1 << n;
51
52          for (int i = 1; i < total_steps; i++)
53          {
54              // n=8
55              // p_index = 1
56              // B[1]
57              // B=00000000
58              // B=01000000
59              int p_index = Q(i) - 1;
60              B ^= 1 << p_index;
61              target.push_back(B);
62          }
63      }
64
65      void printBinary(int code, int bitrate)
66      {
67          for (int i = bitrate - 1; i >= 0; i--)
68          {
69              cout << ((code >> i) & 1);
70          }
71      }
72
73      void printMultiSet(MultiSet &ms, const vector<int> &u, int bitrate)
74      {
75          if (ms.size() == 0)
76          {
77              cout << "Мультимножество пустое!\n";
78          }
79
80          size_t limit = u.size();
81          bool is_truncated = false;
82
83          if (u.size() > 32)
84          {
85              cout << "\Множествоn слишкомбольшоедлявывода, будутвыведеныпервые32 элемента:\n";
86              limit = 32;
87              is_truncated = true;
```

```
88          }
89
90          bool first = true;
91          cout << "{";
92              for (size_t i = 0; i < limit; ++i)
93              {
94                  if (ms[i] > 0)
95                  {
96                      if (!first)
97                      cout << ", ";
98                      cout << "'";
99                      printBinary(u[i], bitrate);
100                     cout << "': " << ms[i];
101                     first = false;
102                 }
103             }
104
105             if (is_truncated)
106             {
107                 if (!first)
108                 cout << ", ...";
109                 else
110                 cout << "...";
111             }
112             cout << "}\n";
113     }
114
115     long long createUniverse(MultiSet &u)
116     {
117         srand(time(nullptr));
118         u.resize(U_SIZE, 0);
119         if (U_SIZE == 0)
120         return 0;
121         long long total_capacity = 0;
122         for (int i = 0; i < U_SIZE; ++i)
123         {
124             u[i] = 1 + rand() % 10; // случайноечислоот1 до10
125             total_capacity += u[i];
126         }
127         return total_capacity;
128     }
129
130     void fillManually(MultiSet &ms, const vector<int> &universe, int bitrate, int
        target_capacity)
131     {
132         int current_capacity = 0;
```

```
133
134         cout << "\Доступныеn элементыуниверсумакод( имаксимальнаякратностьвU):\n";
135         for (int i = 0; i < universe.size(); ++i)
136         {
137             cout << i + 1 << ". [";
138             printBinary(universe[i], bitrate);
139             cout << "] -кратностьвU: " << multi_universe[i] << "\n";
140         }
141
142         while (current_capacity < target_capacity)
143         {
144             cout << "\Текущаяn мощность: " << current_capacity << " из" << target_capacity <<
                    "\n";
145             int idx = getValidatedInt("Выберите номерэлементадлядобавлениянумерация( от1): ",
                    1, universe.size()) - 1;
146
147             int remaining_in_u = multi_universe[idx] - ms[idx];
148             if (remaining_in_u <= 0)
149             {
150                 cout << "Этот элементнедоступеноставшаяся( ёмкостьвуниверсумеравна0).
                        Выберитедругойэлемент.\n";
151                 continue;
152             }
153
154             int max_allowed = min(remaining_in_u, target_capacity - current_capacity);
155             string prompt = format("Введите кратностьот1 до{} оставшаяся( ёмкостьвU: {}): ",
                    max_allowed, remaining_in_u);
156             int count = getValidatedInt(prompt, 1, max_allowed);
157
158             ms[idx] += count;
159             current_capacity += count;
160         }
161     }
162
163     void fillAutomatically(MultiSet &ms, const vector<int> &universe, int target_capacity)
164     {
165         int current_capacity = 0;
166         while (current_capacity < target_capacity)
167         {
168             int idx = rand() % U_SIZE;
169             if (ms[idx] < multi_universe[idx])
170             {
171                 ms[idx]++;
172                 current_capacity++;
173             }
174         }
```

```
175     }
176
177     int main(int argc, char const *argv[])
178     {
179         // приbitrate = 28 времявычислениякодагреясоставиловсреднем7.3 секунды. при27 - 3.6
                сек.
180         int bitrate = getValidatedInt("Введите разрядностькодаГрея(0-27): ", 0, 27);
181         auto start = chrono::steady_clock::now();
182         MultiSet A, B;
183         vector<int> universe;
184         if (bitrate == 0)
185         {
186             cout << "\Универсумn пустмощность( равна0). МножестваA иB
                    автоматическиоставленыпустыми.\n";
187         }
188         else
189         {
190             universe.resize(1 << bitrate, 0);
191             generateGrayCode(bitrate, universe);
192             U_SIZE = universe.size();
193
194             A.resize(U_SIZE, 0);
195             B.resize(U_SIZE, 0);
196
197             long long total_u_capacity = createUniverse(multi_universe);
198
199             auto end = chrono::steady_clock::now();
200             chrono::duration<double> elapsed = end - start;
201             cout << "Количество кодов: " << U_SIZE << ". Прошловремени: " << elapsed.count()
                    << " сек.\n";
202
203             cout << "\Сгенерированныйn универсумU: ";
204             printMultiSet(multi_universe, universe, bitrate);
205
206             int lower_capacity = 0;
207             int upper_capacity;
208             if (100LL < total_u_capacity)
209             upper_capacity = 100;
210             else
211             upper_capacity = total_u_capacity;
212
213             int capacityA = getValidatedInt(format("\Введитеn мощностьмультимножестваA от( {}
                     до{}): ", lower_capacity, upper_capacity),
214             lower_capacity, upper_capacity);
215             int capacityB = getValidatedInt(format("\Введитеn мощностьмультимножестваB от( {}
                     до{}): ", lower_capacity, upper_capacity),
```

```
216             lower_capacity, upper_capacity);
217
218             if (capacityA > 0)
219             {
220                 int choiceA = getValidatedInt("\Выберитеn способзаполнениямножестваA (1 -
                        Вручную, 2 - Автоматически): ", 1, 2);
221                 if (choiceA == 1)
222                 fillManually(A, universe, bitrate, capacityA);
223                 else
224                 fillAutomatically(A, universe, capacityA);
225             }
226             if (capacityB > 0)
227             {
228                 int choiceB = getValidatedInt("\Выберитеn способзаполнениямножестваB (1 -
                        Вручную, 2 - Автоматически): ", 1, 2);
229                 if (choiceB == 1)
230                 fillManually(B, universe, bitrate, capacityB);
231                 else
232                 fillAutomatically(B, universe, capacityB);
233             }
234         }
235
236         while (true)
237         {
238             cout << "\n\n================ Главноеменю================\n";
239             cout << "0. Выход\n";
240             cout << "1. Показатьисходныемультимножества\n";
241             cout << "2. Объединение(A u B)\n";
242             cout << "3. Пересечение(A n B)\n";
243             cout << "4. Разность(A \\ B иB \\ A)\n";
244             cout << "5. Симметрическаяразность(A /\\ B)\n";
245             cout << "6. Дополнение(U \\ A иU \\ B)\n";
246             cout << "7. Арифметическаясумма\n";
247             cout << "8. Арифметическаяразность\n";
248             cout << "9. Арифметическоепроизведение\n";
249             cout << "10. Арифметическоеделение\n";
250             cout << "============================================\n";
251
252             int menuChoice = getValidatedInt("Выберите действие: ", 0, 10);
253
254             if (menuChoice == 0)
255             {
256                 cout << "Программа завершена.\n";
257                 break;
258             }
259
```

```
260             MultiSet result;
261 
262             switch (menuChoice)
263             {
264                 case 1:
265                 cout << "\Мультимножествоn A: ";
266                 printMultiSet(A, universe, bitrate);
267                 cout << "Мультимножество B: ";
268                 printMultiSet(B, universe, bitrate);
269                 cout << "Универсум: ";
270                 printMultiSet(multi_universe, universe, bitrate);
271                 break;
272                 case 2:
273                 result = uni(A, B);
274                 cout << "Объединение (A u B): ";
275                 printMultiSet(result, universe, bitrate);
276                 break;
277                 case 3:
278                 result = insertion(A, B);
279                 cout << "Пересечение (A n B): ";
280                 printMultiSet(result, universe, bitrate);
281                 break;
282                 case 4:
283                 result = difference(A, B);
284                 cout << "Разность (A \\ B): ";
285                 printMultiSet(result, universe, bitrate);
286 
287                 result = difference(B, A);
288                 cout << "Разность (B \\ A): ";
289                 printMultiSet(result, universe, bitrate);
290                 break;
291                 case 5:
292                 result = sym_difference(A, B);
293                 cout << "Симметрическая разность(A /\\ B): ";
294                 printMultiSet(result, universe, bitrate);
295                 break;
296                 case 6:
297                 result = complement(A);
298                 cout << "Дополнение (U \\ A): ";
299                 printMultiSet(result, universe, bitrate);
300 
301                 result = complement(B);
302                 cout << "Дополнение (U \\ B): ";
303                 printMultiSet(result, universe, bitrate);
304                 break;
305                 case 7:
```

```
306             result = A + B;
307             cout << "Арифметическая сумма(A + B): ";
308             printMultiSet(result, universe, bitrate);
309             break;
310             case 8:
311             result = A - B;
312             cout << "Арифметическая разность(A - B): ";
313             printMultiSet(result, universe, bitrate);
314 
315             result = B - A;
316             cout << "Арифметическая разность(B - A): ";
317             printMultiSet(result, universe, bitrate);
318             break;
319             case 9:
320             result = A * B;
321             cout << "Арифметическое произведение(A * B): ";
322             printMultiSet(result, universe, bitrate);
323             break;
324             case 10:
325             result = A / B;
326             cout << "Арифметическое деление(A / B): ";
327             printMultiSet(result, universe, bitrate);
328 
329             result = B / A;
330             cout << "Арифметическое деление(B / A): ";
331             printMultiSet(result, universe, bitrate);
332             break;
333         }
334     }
335 
336     return 0;
337 }
```

Приложение Г. Скрипт сборки CMakeLists.txt

Листинг 4: Скрипт сборки CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
set (CMAKE_CXX_STANDARD 20)
project(lab1 LANGUAGES CXX)

set(CMAKE_CXX_FLAGS "-fsanitize=address -fno-omit-frame-pointer -O1")

add_library(mylib STATIC
src/MultiSet.hpp
src/MultiSet.cpp
)

add_executable(main
src/main.cpp
)

target_link_libraries(main mylib)

file(GENERATE
OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/run_test.sh"
CONTENT "cat ../test_data.txt | ./main\n"
)
```